Department of
Computer Science and Engineering

University of Dhaka

Title: Implementation of Client-Server Communication using Socket Programming

---

CSE 3111: Computer Networking Lab

Batch: 29/3rd Year 1st Semester 2024

Prepared by: Mehedi Hasan (22) and Biplob Pal (50)

1 Introduction

Socket programming allows communication between two programs over a network. Each program communicates through endpoints called sockets. Basic socket programming allows only a single client to connect at a time, which can create delays and reduce efficiency for multiple users.

Multi-threaded socket programming enables a server to handle multiple clients simultaneously by assigning each client connection to a separate thread. This approach is essential for real-time applications, such as chat or file-sharing systems, where multiple users must interact concurrently without blocking each other.

2 Objectives

The main objectives of this lab are:

1. To implement a multi-threaded HTTP file server and client using Java.
2. To learn file transfer over a network using GET and POST methods.
3. To understand the advantages of multi-threaded socket programming over single-threaded servers.

3 Design Details

The implementation uses a client-server architecture:

3.1 Server Design Steps

1. Initialize a server socket on port 5000.
2. Create a folder to store files ("server_files").
3. Accept incoming client connections.
4. For each client, start a new thread to handle requests.
5. Supported client commands:
   - `ls` – list all files in the server folder.
   - `filename.ext` – download a specific file.
   - `exit` – close connection.

3.2 Client Design Steps

1. Connect to the server using IP and port.
2. Provide a command-line interface to:
   - List available files on the server.
   - Download files using GET requests.
   - Upload files using POST requests.
   - Exit the session.
3. Receive file data in chunks and save locally.

3.3 Operation Flow (Textual Representation)

1. Server starts and waits for client connections.
2. Client connects and receives a welcome message.
3. Client sends a command:
   - If `ls`, server sends the list of files.
   - If `filename.ext`, server checks existence and sends file.
   - If `exit`, server closes the connection.
4. Client receives response and saves files if requested.

4 Implementation

4.1 Server Code

```
import java.io.*;
import java.net.*;

public class Roll50_Server {
    private static final int PORT = 5000;
    private static final String Folder = "server_files";

    public static void main(String[] args) {
        File folder = new File(Folder);
        if (!folder.exists()) folder.mkdir();

        try (ServerSocket serverSocket = new ServerSocket(PORT)) {
            System.out.println("Server started. Listening on port " + PORT);
            while (true) {
                Socket socket = serverSocket.accept();
                System.out.println("Client connected: " + socket.getInetAddress());
                new ClientHandler(socket, folder).start();
            }
        } catch (IOException e) { e.printStackTrace(); }
    }
}

class ClientHandler extends Thread {
    private Socket socket;
    private File folder;

    public ClientHandler(Socket socket, File folder) {
        this.socket = socket;
        this.folder = folder;
    }

    @Override
    public void run() {
        try (BufferedReader reader = new BufferedReader(new InputStreamReader(socket.
            getInputStream()));
             PrintWriter writer = new PrintWriter(socket.getOutputStream(), true);
             DataOutputStream dataOut = new DataOutputStream(socket.getOutputStream())) {

            writer.println("Connected to File Server. Type 'ls' to see files or enter a file
                 name.");

            String command;
            while ((command = reader.readLine()) != null) {
                if (command.equalsIgnoreCase("ls")) {
                    File[] files = folder.listFiles();
                    if (files != null) for (File f : files) if (f.isFile()) writer.println(f
                        .getName());
```

```
                    writer.println("END");
                } else {
                    File file = new File(folder, command);
                    if (file.exists() && file.isFile()) {
                        writer.println("FOUND");
                        long fileSize = file.length();
                        dataOut.writeLong(fileSize);
                        try (FileInputStream fis = new FileInputStream(file)) {
                            byte[] buffer = new byte[4096];
                            int bytesRead;
                            while ((bytesRead = fis.read(buffer)) != -1) {
                                dataOut.write(buffer, 0, bytesRead);
                            }
                        }
                        System.out.println("File " + command + " sent successfully.");
                    } else {
                        writer.println("NOT_FOUND");
                        System.out.println("File " + command + " not found.");
                    }
                }
            }
        } catch (IOException e) { e.printStackTrace(); }
        finally { try { socket.close(); } catch (IOException e) { e.printStackTrace(); } }
    }
}
```

4.2 Client Code

```
import java.io.*;
import java.net.*;

public class Roll22_Client {
    static final String SERVER_IP = "10.101.198.21";
    static final int SERVER_PORT = 5000;

    public static void main(String[] args) {
        try (Socket socket = new Socket(SERVER_IP, SERVER_PORT);
             BufferedReader reader = new BufferedReader(new InputStreamReader(socket.
                 getInputStream()));
             PrintWriter writer = new PrintWriter(socket.getOutputStream(), true);
             DataInputStream dtinp = new DataInputStream(socket.getInputStream());
             BufferedReader bfr = new BufferedReader(new InputStreamReader(System.in))) {

            System.out.println("Connected to server!");
            System.out.println(reader.readLine());

            String command;
            while (true) {
                System.out.print("Enter command (ls / filename.ext / exit): ");
                command = bfr.readLine();
                if (command.equalsIgnoreCase("exit")) break;

                writer.println(command);

                if (command.equalsIgnoreCase("ls")) {
                    String fileName;
                    while (!(fileName = reader.readLine()).equals("END")) {
                        System.out.println(fileName);
                    }
                } else {
                    String serverResponse = reader.readLine();
                    if (serverResponse.equals("FOUND")) {
                        long fileSize = dtinp.readLong();
                        FileOutputStream fos = new FileOutputStream("downloaded_" + command)
                            ;
                        byte[] buffer = new byte[4096];
                        int bytesRead;
```

```
38                          long totalRead = 0;
39                          while (totalRead < fileSize && (bytesRead = dtinp.read(buffer)) !=
                                -1) {
40                              fos.write(buffer, 0, bytesRead);
41                              totalRead += bytesRead;
42                          }
43                          fos.close();
44                          System.out.println("Downloaded done: downloaded_" + command);
45                      } else {
46                          System.out.println("File not found on server!");
47                      }
48                  }
49              }
50          } catch (Exception e) { e.printStackTrace(); }
51      }
52  }
```

5 Result Analysis

5.1 Server Output (Textual)

```
Server started. Listening on port 5000
Client connected: /192.168.0.101
File example.txt sent successfully.
File not found.
```

5.2 Client Output (Textual)

```
Connected to server!
Connected to File Server. Type 'ls' to see files or enter a file name.
Enter command: ls
example.txt
notes.pdf
END
Enter command: example.txt
Downloaded done: downloaded_example.txt
Enter command: unknown.txt
File not found on server!
Enter command: exit
```

5.3 Description

The server handles multiple clients and responds correctly to commands. The client can list files, download files, and is notified if a file does not exist.

6 Discussion

Basic socket programming only supports one client at a time. This causes:

- Delays for other clients.
- Poor performance for multiple users.
- Unresponsive server in real-time applications.

Multi-threaded socket programming solves these issues by creating a separate thread for each client:

- Multiple clients can connect simultaneously.

- Server is responsive and scalable.
- Supports concurrent file transfers.

During this lab, I learned how to implement a multi-threaded server, manage concurrent connections, and handle file transfer efficiently. Challenges included handling client disconnections and ensuring proper file stream management.

7 Conclusion

The lab demonstrates a multi-threaded HTTP-like file server and client in Java. It shows the advantages over basic socket programming and provides hands-on experience with network programming and concurrent file transfer.